Homework Set 1 – Hardware Architecture

Due Date: Monday, Apr 6, 2026

Problem 1: Number Types and Ranges. [15 points]

1. [6 points] Assume A and B are both 32-bit signed integers. Recall that signed integers are represented in their 2's complements, and the most significant bit is the sign bit. Prove the following **necessary and sufficient** conditions to detect overflow of $A + B$: let $[A + B]$ be the result of $A + B$ represented in 32 bits (bit 31 to bit 0), i.e., after throwing away bit 32. Overflow occurs if and only if $A > 0$, $B > 0$, $[A + B] < 0$; or $A < 0$, $B < 0$, $[A + B] \geq 0$.

2. [9 points] Identify the bugs in the following pieces of C code. First explain why the code does not follow the expected behavior. Then provide the correct code with minimum changes.

a)

```
// Loop over an array in the inverse order
for (unsigned int i = 9; i >= 0; i--) {
  printf("data %u = %d\n", i, a[i]);
}
```

b)

```
// Check value range; output 1 if and only if value is > 10
constexpr unsigned int lim = 10;
int check_data(int val) {
  return val > lim ? 1 : 0;
}
```

c)

```
// Check if buffer has enough space to keep data
void copy_in(void* buffer, int buffer_size,
       void* data, unsigned int data_size) {
  if (buffer_size – data_size >= 0) memcpy(buf, data, data_size);
}
```

Hint: What happens when you mix signed and unsigned data in an expression?

Problem 2: Assembly Code. [15 points]

1. [4 points] Write short RISC-V assembly sequences with no more than 3 instructions for the following functionalities. Assume a 32-bit machine and `sizeof(int) == 4`. Your answers should have no other side efforts such as modifying other registers and memory addresses.

a) Get the value of `A[3][4]` into register `x2`, where `A` is a multi-dimensional array `int A[8][8]`, whose starting address is stored in register `x1`.
b) Get the value of `A[3][4]` into register `x2`, where `A` is a multi-level array with element type as `int`, and the starting address of its first level is stored in register `x1`.

2. [3 points] On a 64-bit RISC-V machine, assume the standard LP64 data model and natural alignment. Choose the correct assembly code to load `nptr->val` into register `x4`, where `nptr` is a pointer to a linked list node defined as below, and its value (i.e., the address of the node it points to) is stored in register `x3`.

```
struct Node {
  char    flag;
  int32_t val;
  Node*   next;
  short   tag;
};
```

a) `lw x4, 1(x3)`
b) `lw x4, 2(x3)`
c) `lw x4, 4(x3)`
d) `lw x4, 8(x3)`

If then in a function, we declare an array using `Node arr[10];`, how many bytes do we consume on the heap and the stack, respectively?

Now suppose we want to minimize the total size of struct Node by reordering its fields. Give one such ordering. Under your reordered layout, how many bytes would `Node arr[10];` allocate on the heap and on the stack, respectively?

3. [4 points] Write out the C code that is functionally equivalent to the following RISC-V assembly.

```
        j  C
L:      slli x4, x1, 2
        add  x4, x4, x2
        lw   x4, 0(x4)
        add  x3, x3, x4
        addi x1, x1, -1
C:      bne  x1, x0, L
```

4. [4 points] Fix the following buggy assembly code, which tries to calculate in register x3 the maximum value of an array of 32-bit integers, whose starting address is stored in register x1. The total number of integers in the array is stored in register x2. You may assume x2>0. You should not increase the number of instructions in this code.

```
     lw   x3, 0(x1)
     addi x4, x0, 0
L:   beq  x4, x2, E
     lw   x5, 0(x1)
     blt  x3, x5, U
     add  x3, x5, x0
U:   addi x4, x4, 1
     addi x1, x1, 1
     j    L
E:
```

Problem 3: Processor Performance. [20 points]

1. [10 points] With a fixed budget of chip area, now we would like to decide between two processor architectures: 1) one big and faster core, vs. 2) N parallel small cores but each is slower. As you may expect, which architecture is better would depend on the available parallelism in the programs.

Assume the total area is A. When we use a certain amount of area a to implement a core, the core offers a performance (in terms of number of instructions executed per second) proportional to the square root of the area, e.g., $\eta\sqrt{a}$ where η is a constant coefficient.

Further assume that the program is composed of serial and perfectly parallel portions. The serial portion can only use a single core (no matter big or small), and the parallel portion can be executed on as many cores as available, and enjoy a linear speedup with respect to the number of cores.

Under such scenario, for different values of N, calculate the minimum fraction of the parallel portion, f, that makes the configuration with multiple small cores perform better than that with a single big core. Fill in the table below. You must show your **calculation process details** in addition to the table.

Number of small cores N	**Parallel portion f**
2	
4	
8	

2. [4 points] Now assume a given program has 70% parallel portion, i.e., $f = 70\%$. What is the speedup (or slowdown) of 8 small cores vs. 1 big core? The speedup of A vs. B is defined as the execution time of B / the execution time of A.

3. [6 points] With the same program as in part 2, consider a hybrid design where we use half of the chip area to build a big core, and the rest half to build 4 small cores. Assume the serial portion runs on the big core, and the parallel portion **only** runs on the 4 small cores. What are the speedups this hybrid design can achieve, compared to the two configurations in part 2, respectively?

Problem 4: Cache organization and performance. [30 points]

In this problem, use 1k = 1024, 1M = 2^{20}, 1G = 2^{30}, and so on.

1. [9 points] Assume we would like to build a cache to be able to store 32 kB of data in a system with 48-bit memory address. Each entry contains a valid bit, a tag, and the data block, as we discussed in the lecture. Ignore other metadata fields. For each of the following configurations, how many SRAM bits do you need to build such a cache? You need to include all data, tags, and valid bits in your result, because all these fields need to use SRAM to implement. You can represent your results in kbits.

a) 64-byte block size, 4-way set associative.
b) 8-byte block size, 4-way set associative.
c) 64-byte block size, 16-way set associative.

2. [4 points] In the lecture we claim that using a multi-level hierarchy of caches is better than using a single cache. Compare the AMAT values of the following three configurations to verify the above conclusion. The memory access latency is 100 ns in all cases. All cases run the same program.

a) One level of a small cache, hit latency = 1 ns, hit rate = 60%.
b) One level of a large cache, hit latency = 20 ns, hit rate = 90%.
c) Two levels, with the first level the same as a), and the second level the same as b). You will need to first figure out the hit rates for the two levels based on known information.

3. [4 points] Consider a full system with a processor, a single cache, and an off-chip memory. We execute different programs, i.e., sequences of instructions, on this system. The processor follows the simple design we discussed in the lecture, i.e., sequentially executing each instruction before moving to the next one. All instructions except loads and stores (i.e., memory access instructions) need 1 unit time (i.e., 1 processor clock cycle). Memory access instructions, if hitting in the cache, also cost 1 unit time. The off-chip memory has a latency of 100 unit time.

We are interested in the slowdown caused by the memory hierarchy, which is measured as the ratio of the execution time on the real system vs. the execution time on an unrealistic perfect memory hierarchy with 100% hit rate. Prove that, for any program, this slowdown depends linearly on the ratio of its cache misses and instruction count.

The above ratio is usually called MPKI, i.e., misses per kilo instructions. Based on your proof, MPKI is usually a better metric than miss rate to analyze performance.

4. [10 points] Now we would like to predict the cache behavior of C code for potential code optimization. Given the following code:

```
int x[2][128];
int sum = 0;
for (int i = 0; i < 128; i++) {
  sum += x[0][i] * x[1][i];
}
```

Assume `sizeof(int) == 4`, array x begins at memory address 0x10000000 and is in row-major order. The only memory accesses are to the entries of x, and all other variables are in registers.

Given the following cache configurations, estimate the miss rates. The cache is initially empty in each case.

a) The cache is 512 bytes, direct-mapped, with 16-byte block size.

b) The cache capacity is doubled to 1024 bytes from a). Other parameters do not change.

c) The cache is 512 bytes, 2-way set associative using an LRU policy, and with 16-byte block size.

d) In c), will a larger cache size help to reduce the miss rate? Why or why not?

e) In c), will a larger block size help to reduce the miss rate? Why or why not?

5. [3 points] Given a set-associativity cache, is an LRU replacement policy (i.e., choosing the least recently used block to evict in the set) always better than a random policy (i.e., randomly choosing one block in the set to evict)? If yes, try to prove your conclusion; if not, please provide a counter-example.

Problem 5: Processes and Scheduling. [20 points]

1. [10 points] Explain whether the output sequences are valid with the given piece of code.

a)

```
printf("L0\n");
if (fork() == 0) {
  printf("L1\n");
  if (fork() == 0) {
    printf("L2\n");
  }
}
printf("Bye\n");
```

Candidate output sequences:

(i)

```
L0
Bye
L1
Bye
Bye
L2
```

(ii)

```
L0
Bye
L1
L2
Bye
Bye
```

(iii)

```
L0
L1
L2
Bye
```

(iv)

```
L0
Bye
L1
L2
```

b)

```
int child_status;
pid_t c1, c2;

printf("S\n");

c1 = fork();
if (c1 == 0) {
  printf("C1\n");
  exit(0);
}

c2 = fork();
if (c2 == 0) {
  printf("C2\n");
  exit(0);
}

printf("P0\n");
waitpid(c1, &child_status, 0);
printf("P1\n");
```

Candidate output sequences:

(i)

```
S
P0
C2
C1
P1
```

(ii)

```
S
C1
P0
P1
C2
```

(iii)

```
S
P0
P1
C1
C2
```

(iv)

```
S
C2
P1
P0
C1
```

2. [10 points] Assume we have four jobs, P_i, $i = 1,2,3,4$, with the execution time of 53, 8, 68, 24, respectively. We compare the three scheduling policies, FCFS, RR (Round Robin), and SJF.

a) Fill in the table below to calculate the **wait time** of each job, as well as the average wait time, under different policies. The "worst FCFS" assumes the arrival order of the four jobs that results in the worst-case average wait time. The time quantum of RR is given by the value of q. All RR policies assume the arrival order of P_1 to P_4. Assume zero context switch overhead.

Hint: For RR, you can manually simulate the scheduling process; or you can write a short program to simulate and automatically calculate the numbers.

Wait time

Policy	P_1	P_2	P_3	P_4	Average
FCFS	0	53	61	129	60.75
Worst FCFS					
SJF					
RR (q=5)					
RR (q=8)					
RR (q=10)					
RR (q=20)					

b) Now assume a context switch overhead of 2 unit time. This overhead is only paid when the next job to run is different from the current running job. Also, the first running job starts at time 0 without a context switch overhead at the very beginning. Re-fill the table below for the RR policies. Does the best time quantum choice change?

Wait time

Policy	P_1	P_2	P_3	P_4	Average
RR (q=5)					
RR (q=8)					
RR (q=10)					
RR (q=20)					